

Mochila Multidimensional para Alocação em Nuvem com Simulated Annealing

Gabriel Kruger Brum¹ João Pedro Rocha da Silva¹

¹Centro de Desenvolvimento Tecnológico
Universidade Federal de Pelotas (UFPel)

- 1 Introdução
- 2 Intratabilidade
- 3 Solução Proposta
- 4 Implementação
- 5 Resultados
- 6 Conclusão

- 1 Introdução
- 2 Instabilidade
- 3 Solução Proposta
- 4 Implementação
- 5 Resultados
- 6 Conclusão

Contexto: Alocação de Recursos em Nuvem

Provedores de nuvem (AWS, Azure, Google Cloud) precisam decidir **quais máquinas virtuais (VMs)** alocar em um servidor físico para maximizar o faturamento.

O servidor tem **limites** em várias dimensões:

- CPU
- Memória RAM
- Armazenamento
- Largura de banda

Objetivo: maximizar o lucro sem estourar nenhum recurso.

Problema da Mochila Multidimensional (MKP)

Esse cenário é modelado como o **Multidimensional Knapsack Problem**:

- **Itens** = VMs a alocar
- **Lucro** = faturamento de cada VM
- **Restrições** = limites de cada recurso
- **Decisão** = incluir ou não cada VM

Formulação

$$\max \sum_{j=1}^n p_j x_j$$

sujeito a:

$$\sum_{j=1}^n r_{ij} x_j \leq c_i$$

$$x_j \in \{0, 1\}$$

- 1 Introdução
- 2 Intratabilidade**
- 3 Solução Proposta
- 4 Implementação
- 5 Resultados
- 6 Conclusão

MKP é NP-difícil

Não se conhece algoritmo que resolva o MKP em tempo polinomial.

Cadeia de redução:

- 1 **Subset-Sum** é NP-completo
- 2 Reduz-se ao **Knapsack 0/1**
- 3 Knapsack 0/1 é caso particular do **MKP**
- 4 Logo, MKP é pelo menos tão difícil quanto o Knapsack

Examples

Com 50 itens: $2^{50} \approx 10^{15}$ combinações.

Métodos exatos são inviáveis $\Rightarrow$ precisamos de **meta-heurísticas**.

- 1 Introdução
- 2 Instabilidade
- 3 Solução Proposta**
- 4 Implementação
- 5 Resultados
- 6 Conclusão

Técnica inspirada no **recozimento de metais**:

- Temperatura **alta** → aceita soluções piores (explora)
- Temperatura **baixa** → aceita só melhorias (converge)
- Resfriamento **gradual** ao longo do tempo

Critério de Metropolis

- Solução melhor? → **aceita sempre**
- Solução pior? → aceita com probabilidade que diminui com a temperatura

Vantagem: consegue escapar de ótimos locais, ao contrário de buscas puramente gulosas.

Geração de Vizinho

- 1 Escolher um item aleatório
- 2 Inverter sua seleção (*bit-flip*)
- 3 Se estava incluído → remover
- 4 Se não estava → adicionar

Heurística de Reparação Gulosa

Se a solução viola alguma restrição:

- 1 Calcular a razão lucro/consumo de cada item selecionado
- 2 Ordenar do pior para o melhor
- 3 Remover os piores até restaurar a viabilidade

Examples

A reparação preserva os itens mais “rentáveis” e descarta os que consomem muitos recursos para pouco lucro.

- 1 Começar com solução vazia (reparada)
- 2 **Enquanto** temperatura $>$ mínima:
 - 1 Para cada iteração do patamar:
 - 1 Gerar vizinho (flip + reparação)
 - 2 Se melhor $\rightarrow$ aceitar
 - 3 Se pior $\rightarrow$ aceitar com probabilidade baseada na temperatura
 - 4 Atualizar melhor solução global se necessário
 - 2 Reduzir temperatura ($T \leftarrow \alpha \cdot T$)
- 3 Retornar melhor solução encontrada

- 1 Introdução
- 2 Instabilidade
- 3 Solução Proposta
- 4 Implementação**
- 5 Resultados
- 6 Conclusão

Implementado em **C++17** com organização modular:

Módulos

- `instance.h/.cpp` – Leitura dos dados
- `solution.h/.cpp` – Solução + reparação
- `sa.h/.cpp` – Simulated Annealing
- `main.cpp` – Interface CLI

Compilação e Execução

- `make` – compilação otimizada
- `make static` – binário portátil
- Modo individual ou **batch**
- Saída formatada em Markdown

Parâmetros Padrão

$T_0 = 1000$, $\alpha = 0.98$, $T_{min} = 0.01$, $L = 1000$ iterações/patamar

- 1 Introdução
- 2 Instabilidade
- 3 Solução Proposta
- 4 Implementação
- 5 Resultados**
- 6 Conclusão

Resultados – mknap1.txt (7 instâncias)

Inst.	Itens	Restr.	Ótimo	SA	Qualidade	Tempo
1	6	10	3.800	3.800	100,00%	235 ms
2	10	10	8.706	8.706	100,00%	290 ms
3	15	10	4.015	4.015	100,00%	323 ms
4	20	10	6.120	6.120	100,00%	406 ms
5	28	10	12.400	12.400	100,00%	447 ms
6	39	5	10.618	10.540	99,27%	564 ms
7	50	5	16.537	16.317	98,67%	683 ms
Média					99,71%	421 ms

- **5 de 7** instâncias resolvidas com solução **ótima exata**
- Qualidade média: **99,71%**
- Todas as soluções **viáveis**

Resultados – mknapcb1.txt (30 instâncias, 100 itens)

Faixa	Qtd.	Lucro Médio	Tempo Médio	Viáveis
1–10	10	23.956	701 ms	10/10
11–20	10	43.136	897 ms	10/10
21–30	10	60.265	775 ms	10/10
Total		42.452	792 ms	30/30

- **100% viáveis** – nenhuma restrição violada em 30 instâncias
- Tempo médio de **792 ms** por instância (100 itens, 5 restrições)
- Reparação gulosa funcionou em todos os casos

37 instâncias testadas

- Qualidade média: **99,71%**
- 5/7 ótimos exatos encontrados
- Tempo máximo: 1,2 segundo
- **100% viáveis**

O que isso demonstra

- SA encontra soluções **quase ótimas**
- Escala bem de 6 a 100 itens
- Reparação gulosa é **robusta**
- Tempo viável para uso prático

- 1 Introdução
- 2 Instabilidade
- 3 Solução Proposta
- 4 Implementação
- 5 Resultados
- 6 Conclusão**

O que foi feito

Resolvedor para o Problema da Mochila Multidimensional usando **Simulated Annealing** + **heurística de reparação gulosa**, aplicado à alocação de VMs em nuvem.

Principais resultados

- Qualidade média de **99,71%** em relação ao ótimo
- Todas as soluções **viáveis** (37/37)
- Execução em **menos de 1,3 segundo** por instância



Obrigado!